University Name
Faculty of Computer Science & Information Technology
Department of Software Engineering

CyberStore

Full-Stack E-Commerce Platform

A Modern Web Application for Gaming Hardware

Final Year Graduation Project Report

Last Cycle – Academic Year 2025/2026

Prepared by:

Student Name
Student ID: XXXXXXXXX

Supervised by:

Dr. Supervisor Name
Department of computer science

March 2026

Abstract

CyberStore is a modern, full-stack e-commerce web application designed and developed as a final year graduation project. The platform provides a complete online shopping experience for gaming hardware and computer components, featuring a cyberpunk-themed user interface with full bilingual support (English and Arabic).

The system is built using a three-tier architecture: a React.js frontend with Tailwind CSS for responsive UI, a Node.js/Express.js RESTful API backend, and a MySQL relational database. Key features include user authentication with JWT, product catalog with advanced filtering and search, shopping cart management, order processing, supplier management, product reviews and ratings, an admin dashboard for full CRUD operations, and internationalization (i18n) support.

This report documents the complete software development lifecycle of the project, covering requirements analysis, system design, implementation details, database schema, API architecture, and the technologies employed.

Keywords: E-Commerce, React.js, Node.js, Express.js, MySQL, REST API, JWT Authentication, Tailwind CSS, Zustand, Internationalization, Full-Stack Web Development.

Contents

Abstract **1**

1 Introduction **5**
- 1.1 Project Overview . . . 5
- 1.2 Problem Statement . . . 5
- 1.3 Project Objectives . . . 5
- 1.4 Project Scope . . . 6
- 1.5 Report Structure . . . 6

2 Literature Review and Related Work **7**
- 2.1 Overview . . . 7
- 2.2 Key Concepts . . . 7
  - 2.2.1 E-Commerce Platforms . . . 7
  - 2.2.2 React.js and Single-Page Applications . . . 7
  - 2.2.3 RESTful APIs with Express.js . . . 7
- 2.3 Comparison with Related Work . . . 8

3 System Analysis and Requirements **9**

3.1 Functional Requirements . . . 9
3.1.1 User Management . . . 9
3.1.2 Product Management . . . 9
3.1.3 Shopping Cart and Orders . . . 9
3.1.4 Supplier Management . . . 9
3.1.5 Internationalization . . . 9
3.2 Non-Functional Requirements . . . 9
3.3 Use Cases . . . 10
3.3.1 Use Case Diagram Description . . . 10
3.3.2 Use Case Diagram . . . 11

4 System Design and Architecture **12**
4.1 System Architecture . . . 12
4.2 Frontend Architecture . . . 12
4.3 Backend Architecture . . . 12
4.4 Class Diagram . . . 13
4.5 Directory Structure . . . 14

5 Implementation and Technologies **16**
5.1 Technology Stack . . . 16
5.2 Authentication and Security . . . 16
5.2.1 JWT Authentication Flow . . . 16
5.2.2 Security Measures . . . 17
5.3 State Management with Zustand . . . 17
5.4 Internationalization (i18n) . . . 17
5.5 Frontend Routing . . . 18

6 Database Design **19**
6.1 Database Overview . . . 19
6.2 Entity-Relationship Description . . . 19
6.3 Table Schemas . . . 19
6.4 Indexes and Performance . . . 21
6.5 Auto-Migration . . . 21

7 API Design **22**
7.1 API Overview . . . 22
7.2 Sequence Diagrams . . . 22
7.2.1 Main Sequence Diagram . . . 22
7.2.2 User Registration Flow . . . 23
7.2.3 Order Placement Flow . . . 23
7.2.4 Product Review Submission . . . 24
7.2.5 Admin Product Management . . . 25
7.3 API Endpoints . . . 26
7.4 Request/Response Examples . . . 27
7.5 Error Handling . . . 28

8 User Interface **29**
8.1 Design Philosophy . . . 29
8.2 Key Pages . . . 29

8.2.1 Home Page . . . 29
8.2.2 Products Page . . . 29
8.2.3 Product Detail Page . . . 29
8.2.4 Shopping Cart . . . 30
8.2.5 Admin Dashboard . . . 30
8.2.6 Supplier Pages . . . 30
8.3 Navigation . . . 30
8.4 Footer . . . 30

9 Testing and Evaluation **31**
9.1 Testing Approach . . . 31
9.1.1 API Testing . . . 31
9.1.2 Database Testing . . . 31
9.1.3 Manual Testing . . . 31
9.2 Test Results Summary . . . 32
9.3 Performance Evaluation . . . 32

10 Conclusion and Future Work **33**
10.1 Conclusion . . . 33
10.2 Challenges Faced . . . 33
10.3 Future Work . . . 33

References **35**

List of Figures

3.1 Use Case Diagram . . . 11

4.1 Three-Tier System Architecture . . . 12
4.2 Class Diagram – Backend Models and Relationships . . . 14

7.1 Sequence Diagram – PC Store Main Flow . . . 22
7.2 Sequence Diagram – User Registration Flow . . . 23
7.3 Sequence Diagram – Order Placement Flow . . . 24
7.4 Sequence Diagram – Product Review Submission . . . 25
7.5 Sequence Diagram – Admin Product Creation . . . 26

List of Tables

2.1 Comparison with Related E-Commerce Projects . . . 8

3.1 Key Use Cases 10

5.1 Technology Stack Summary 16
5.2 Application Routes 18

6.1 Database Table Schemas 19

7.1 API Endpoint Reference 26

9.1 Testing Results Summary 32

Chapter 1 Introduction

1.1 Project Overview

CyberStore is a comprehensive e-commerce web application built as a graduation project to demonstrate proficiency in modern full-stack web development. The platform specializes in gaming hardware and computer components, providing a sleek cyberpunk-themed user interface that enhances the shopping experience.

The application follows a client–server architecture, with a decoupled React.js single-page application (SPA) communicating with a Node.js/Express.js RESTful API, backed by a MySQL relational database management system.

1.2 Problem Statement

Traditional e-commerce platforms often suffer from:

- Lack of bilingual support (especially Arabic RTL layout)
- Poor admin management interfaces for products, orders, users, and suppliers
- Absence of supplier relationship management integrated within the e-commerce workflow
- Slow, non-responsive user interfaces that do not leverage modern frontend frameworks
- Lack of real-time product reviews and rating systems

CyberStore addresses these challenges by providing a modern, performant, and feature-rich e-commerce solution with full bilingual support and comprehensive admin tools.

1.3 Project Objectives

The main objectives of this project are:

1. Design and develop a full-stack e-commerce web application using modern technologies.
2. Implement a responsive, accessible, and visually appealing user interface.
3. Build a secure RESTful API with JWT-based authentication and role-based access control.
4. Design a normalized relational database schema for products, orders, users, suppliers, reviews, and categories.
5. Implement full CRUD operations for an admin dashboard.
6. Support bilingual content (English and Arabic) with RTL layout switching.
7. Integrate a supplier management system linked to products.
8. Implement product reviews and ratings.

1.4 Project Scope

The project scope includes:

- **Frontend:** 20+ pages/views including Home, Products, Product Detail, Cart, Checkout, Profile, Login, Register, Categories, Suppliers, Supplier Detail, and Admin pages.
- **Backend:** RESTful API with 7 route groups (auth, products, orders, users, admin, stats, suppliers).
- **Database:** 7 tables (users, categories, suppliers, products, orders, order_items, reviews).
- **Admin Panel:** Dashboard with statistics, product management, order management, user management, and supplier management.
- **i18n:** Full English and Arabic translations (1500+ translation keys).

1.5 Report Structure

This report is organized as follows:

- **Chapter 2:** Literature Review and Related Work
- **Chapter 3:** System Analysis and Requirements
- **Chapter 4:** System Design and Architecture
- **Chapter 5:** Implementation and Technologies
- **Chapter 6:** Database Design
- **Chapter 7:** API Design
- **Chapter 8:** User Interface
- **Chapter 9:** Testing and Evaluation
- **Chapter 10:** Conclusion and Future Work

Chapter 2 Literature Review and Related Work

2.1 Overview

This chapter reviews the key concepts and technologies relevant to the development of CyberStore, and compares the project with existing e-commerce solutions.

2.2 Key Concepts

2.2.1 E-Commerce Platforms

E-commerce platforms enable online transactions between businesses and consumers. Successful platforms require secure authentication, product catalogs with search capabilities, shopping cart workflows, order management, and responsive design. Examples include Shopify, WooCommerce, and Amazon.

2.2.2 React.js and Single-Page Applications

React.js, developed by Meta, is a component-based JavaScript library for building single-page applications (SPAs). SPAs load one HTML page and dynamically update content without full page reloads, resulting in faster and more interactive user experiences.

2.2.3 RESTful APIs with Express.js

REST (Representational State Transfer) uses standard HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources. Express.js is the most widely used Node.js framework for building RESTful APIs due to its simplicity and flexibility.

2.3 Comparison with Related Work

Table 2.1: Comparison with Related E-Commerce Projects

Feature	**ProShop (MERN)**	**Shopify Starter**	**CyberStore (Ours)**
Frontend	React + Bootstrap	Liquid Templates	React + Tailwind CSS
Backend	Node/Express	Ruby on Rails	Node/Express
Database	MongoDB	PostgreSQL	MySQL
i18n Support	No	Partial	Full (EN/AR + RTL)
Supplier Mgmt	No	No	Yes
Reviews	Basic	Plugin-based	Built-in
Admin Panel	Basic	Full	Full (5 modules)
Animations	No	Minimal	Framer Motion

CyberStore differentiates itself through full bilingual support with RTL layout, built-in supplier management, and a modern animated UI using Framer Motion—features not commonly found together in similar academic projects.

Chapter 3 System Analysis and Requirements

3.1 Functional Requirements

3.1.1 User Management

- FR-01: Users can register with name, email, and password.
- FR-02: Users can log in and receive a JWT token.
- FR-03: Users can view and update their profile.
- FR-04: Admin users can view, edit, and delete user accounts.

3.1.2 Product Management

- FR-05: Users can browse products with pagination, sorting, and filtering.
- FR-06: Users can search products by name or description (full-text search).
- FR-07: Users can view product details including features, specifications, and reviews.
- FR-08: Admin can create, edit, and delete products.
- FR-09: Products can be assigned to categories and suppliers.
- FR-10: Users can submit reviews and ratings for products.

3.1.3 Shopping Cart and Orders

- FR-11: Users can add/remove products from the cart.
- FR-12: Users can proceed to checkout and place orders.
- FR-13: Users can view their order history.
- FR-14: Admin can view all orders and update order status.

3.1.4 Supplier Management

- FR-15: Public supplier listing and detail pages.
- FR-16: Admin can create, edit, delete, and toggle supplier status.
- FR-17: Suppliers linked to products via foreign key.
- FR-18: Supplier details include contact, address, and financial information.

3.1.5 Internationalization

- FR-19: Full English and Arabic language support.
- FR-20: Automatic RTL/LTR layout switching.
- FR-21: Language toggle accessible from the navigation bar.

3.2 Non-Functional Requirements

- NFR-01: **Performance** – Page load time under 3 seconds.

- NFR-02: **Security** – Passwords hashed with bcrypt; JWT-based auth with expiration.
- NFR-03: **Responsiveness** – Fully responsive design for mobile, tablet, and desktop.
- NFR-04: **Usability** – Intuitive navigation and consistent design patterns.
- NFR-05: **Scalability** – Database connection pooling; modular code structure.
- NFR-06: **Maintainability** – Separation of concerns; component-based architecture.

3.3 Use Cases

3.3.1 Use Case Diagram Description

The PC Store use-case model supports three primary actors:

1. **Visitor (Guest)** – Non-authenticated user who can browse/search products, view product details, register, and login.
2. **Client** – Authenticated user who can manage cart, checkout, place orders, track order status, view order history, and update profile.
3. **Administrator** – Inherits client capabilities and additionally manages products, categories, orders, clients, and reports.

The following UML relationships are modeled in the diagram:

- **«include»:** *Place Order* includes *Manage Cart* and *Login* (when authentication is required).
- **«extend»:** *View Order History* extends *Login*.
- **Generalization:** *Administrator* is a specialization of *Client*.

Table 3.1: Key Use Cases

ID	Actor	Use Case
UC-01	Visitor	Consult product catalog
UC-02	Visitor	Search product by name/category
UC-03	Visitor	View product details (price/specifications)
UC-04	Visitor	Register account
UC-05	Visitor/Client	Login
UC-06	Client	Manage cart (add/update/remove items)
UC-07	Client	Place order (checkout)
UC-08	Client	View order history
UC-09	Client	Track order status
UC-10	Client	Update profile (address/password)
UC-11	Administrator	Manage products (create/update/delete)
UC-12	Administrator	Manage categories
UC-13	Administrator	Manage orders (status/details)
UC-14	Administrator	Manage clients (view/block)
UC-15	Administrator	View reports (sales/stock)

3.3.2 Use Case Diagram

Figure 3.1 illustrates the use case diagram showing the three actors and their interactions with the system.

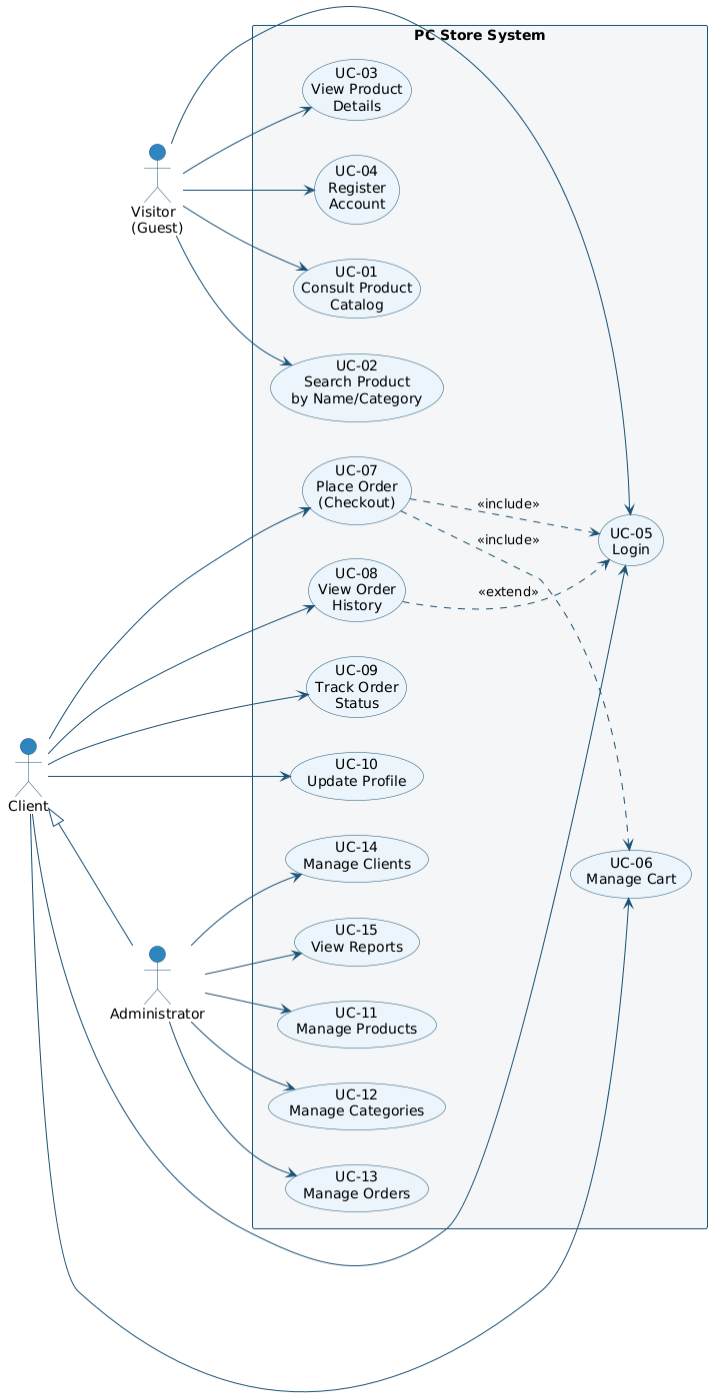

Figure 3.1: Use Case Diagram

Chapter 4 System Design and Architecture

4.1 System Architecture

CyberStore follows a **three-tier architecture**:

1. **Presentation Tier (Frontend):** React.js SPA served on port 5173/3000, communicating with the backend via HTTP/REST.
2. **Application Tier (Backend):** Node.js/Express.js server on port 5000, exposing RESTful API endpoints.
3. **Data Tier (Database):** MySQL 8.x relational database on port 3306.

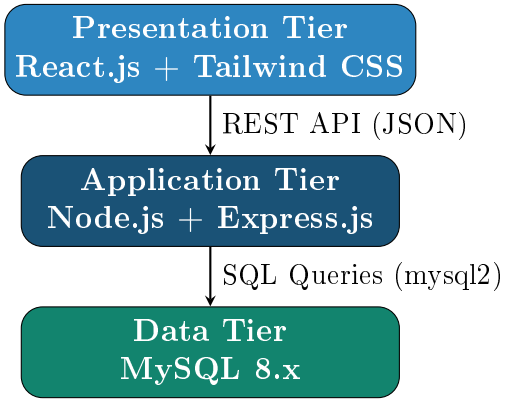

Figure 4.1: Three-Tier System Architecture

4.2 Frontend Architecture

The frontend follows a component-based architecture:

- **Pages (20+):** Home, Products, ProductDetail, Cart, Checkout, Login, Register, Profile, Categories, Suppliers, SupplierDetail, About, Contact, Support, NewArrivals, Trending, Sale, PreOrder, TrackOrder, OrderConfirmation.
- **Admin Pages (8):** Dashboard (with Overview, Products, Orders, Users, Suppliers tabs), AddProduct, AddSupplier, AdminLogin, ProductsManager, OrdersManager, UsersManager, SuppliersManager.
- **Components:** CyberNavbar, ProductCard, Footer, TerminalAnimation.
- **State Management:** Zustand stores – `authStore` (authentication), `cartStore` (shopping cart), `langStore` (language/i18n).
- **Services:** Centralized Axios API client (`api.js`) with interceptors for auth tokens and error handling.

4.3 Backend Architecture

The backend follows the MVC (Model–View–Controller) pattern:

- **Routes (7 modules):** auth, products, orders, users, admin, stats, suppliers.
- **Controllers (6):** authController, productController, orderController, userController, adminController, supplierController, statsController.
- **Models (7):** User, Product, Category, Order, OrderItem, Review, Supplier.
- **Middleware:** JWT authentication (`protect`), admin authorization (`admin`), error handling.
- **Config:** Database connection pool with auto-migration (table creation and schema updates).

4.4 Class Diagram

Figure 4.2 shows the main backend model classes with their attributes, methods, and relationships.

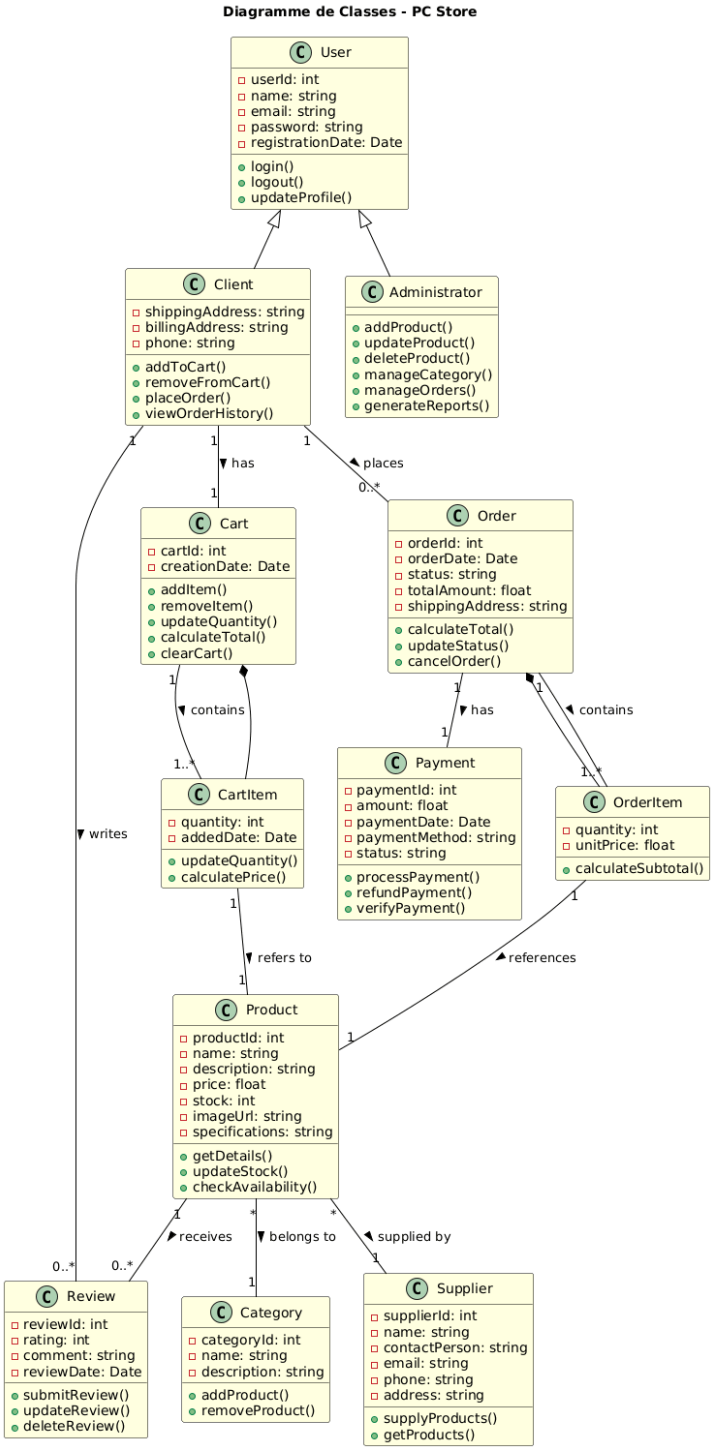

Figure 4.2: Class Diagram – Backend Models and Relationships

4.5 Directory Structure

```
cyberstore/
  +-- backend/
  |    +-- server.js              # Express app entry point
  |    +-- config/db.js           # MySQL pool + auto-migration
```

```
|   +-- controllers/          # Business logic (6 controllers)
|   +-- middleware/            # auth.js (JWT + admin)
|   +-- models/               # Data models (7 models)
|   +-- routes/               # API routes (7 modules)
|   +-- scripts/              # Utility & test scripts
|   +-- public/uploads/       # Uploaded product images
+-- frontend/
|   +-- src/
|   |   +-- App.jsx           # Router + route definitions
|   |   +-- components/       # Reusable UI components
|   |   +-- pages/            # Page components (20+)
|   |   +-- pages/Admin/      # Admin panel (8 components)
|   |   +-- pages/store/      # Zustand stores (3 stores)
|   |   +-- services/api.js   # Axios API client
|   |   +-- i18n/             # translations.js (EN + AR)
|   +-- public/               # Static assets
+-- package.json              # Root scripts (concurrently)
```

Listing 4.1: Project Directory Structure

Chapter 5 Implementation and Technologies

5.1 Technology Stack

Table 5.1: Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js 18.2	Component-based UI library
Routing	React Router DOM 6	Client-side routing and navigation
Styling	Tailwind CSS 3.4	Utility-first CSS framework
Animations	Framer Motion 12	Declarative animations and transitions
Icons	Lucide React	Consistent SVG icon library
State	Zustand 4.4	Lightweight state management
HTTP Client	Axios 1.6	HTTP requests with interceptors
Charts	Recharts 3.7	Dashboard statistics visualization
Backend	Node.js	JavaScript runtime environment
Framework	Express.js 4.18	Minimalist web framework
Database Driver	mysql2 3.6	MySQL client with promise support
Auth	jsonwebtoken 9.0	JWT token generation and verification
Encryption	bcryptjs 2.4	Password hashing
Environment	dotenv 16.3	Environment variable management
Email	Nodemailer 6.10	Email sending capability
PDF	PDFKit 0.13	PDF document generation
Database	MySQL 8.x	Relational database management system
Dev Tools	Nodemon 3.0	Auto-restart on code changes
	Concurrently 8.2	Run frontend + backend simultaneously
	ESLint	Code linting and quality

5.2 Authentication and Security

5.2.1 JWT Authentication Flow

1. User submits credentials (email + password) via `POST /api/auth/login`.
2. Server validates credentials and compares bcrypt-hashed password.
3. On success, server generates a JWT token containing the user ID.
4. Token is returned to the client and stored in `localStorage`.
5. Subsequent API requests include the token in the `Authorization: Bearer <token>` header.
6. The `protect` middleware verifies the token and attaches the user object to the request.
7. The `admin` middleware checks if the user role is `'admin'`.

5.2.2 Security Measures

- Passwords hashed using `bcryptjs` with salt rounds.
- JWT tokens with configurable expiration.
- CORS configuration with whitelisted origins.
- Input validation on all API endpoints.
- SQL injection prevention via parameterized queries (`mysql2` prepared statements).
- Role-based access control (user vs. admin).
- Axios interceptor auto-redirects on 401 responses.

5.3 State Management with Zustand

Three Zustand stores power the application state:

```
// authStore.js - Authentication state
const useAuthStore = create((set) => ({
  user: null,
  token: null,
  isAuthenticated: false,
  login: async (email, password) => { /* ... */ },
  register: async (name, email, password) => { /* ... */ },
  logout: () => { /* ... */ },
  isAdmin: () => { /* checks user.role === 'admin' */ },
}));

// cartStore.js - Shopping cart state
const useCartStore = create(persist((set, get) => ({
  items: [],
  addToCart: (product) => { /* ... */ },
  removeFromCart: (id) => { /* ... */ },
  clearCart: () => { /* ... */ },
  getTotal: () => { /* ... */ },
})));

// langStore.js - Internationalization state
const useLangStore = create(persist((set, get) => ({
  lang: 'en',
  toggleLang: () => { /* switches en <-> ar, updates dir */
    },
  t: (key) => { /* returns translated string */ },
})));
```

Listing 5.1: Zustand Store Architecture

5.4 Internationalization (i18n)

The application supports full English and Arabic translations:

- Over 1,500 translation keys organized by page/section.
- The `langStore` provides a `t(key)` function that retrieves the correct translation.

- RTL/LTR layout automatically switches when the language changes.
- `document.documentElement.dir` and `document.documentElement.lang` are updated.
- All user-facing text across 20+ pages is translated.

5.5 Frontend Routing

React Router DOM v6 manages all client-side routes:

Table 5.2: Application Routes

Path	Access	Component
`/`	Public	Home
`/products`	Public	Products (catalog)
`/products/:id`	Public	ProductDetail
`/categories`	Public	Categories
`/suppliers`	Public	Suppliers
`/suppliers/:id`	Public	SupplierDetail
`/cart`	Public	Cart
`/login`	Public	Login
`/register`	Public	Register
`/about`	Public	About
`/contact`	Public	Contact
`/support`	Public	Support
`/checkout`	Auth	Checkout
`/profile`	Auth	Profile
`/order-confirmation/:id`	Auth	OrderConfirmation
`/admin/login`	Public	AdminLogin
`/admin/dashboard`	Admin	Dashboard
`/admin/products/new`	Admin	AddProduct
`/admin/products/edit/:id`	Admin	AddProduct (edit mode)
`/admin/suppliers/new`	Admin	AddSupplier
`/admin/suppliers/edit/:id`	Admin	AddSupplier (edit mode)

Chapter 6 Database Design

6.1 Database Overview

The application uses MySQL 8.x with the InnoDB engine and `utf8mb4` character set for full Unicode support (including Arabic text and emojis). The schema consists of 7 tables with foreign key constraints and indexed columns for performance.

6.2 Entity-Relationship Description

- A **User** can place many **Orders** (1:N).
- An **Order** contains many **OrderItems** (1:N).
- A **Product** belongs to one **Category** (N:1).
- A **Product** can belong to one **Supplier** (N:1, nullable).
- A **Product** can have many **Reviews** (1:N).
- A **Review** is written by one **User** (N:1).

6.3 Table Schemas

Table 6.1: Database Table Schemas

Table	Column	Type	Description
`users`	id	INT PK AUTO_INC	Primary key
	username	VARCHAR(50) UNIQUE	Display name
	email	VARCHAR(100) UNIQUE	Login email
	password	VARCHAR(255)	Bcrypt hash
	role	ENUM('user','admin')	Access level
	balance	DECIMAL(10,2)	Account balance
	is_active	BOOLEAN	Account status
	created_at	TIMESTAMP	Registration date
`categories`	id	INT PK AUTO_INC	Primary key
	name	VARCHAR(100) UNIQUE	Category name
	slug	VARCHAR(100) UNIQUE	URL-friendly slug
	description	TEXT	Category description
	icon	VARCHAR(50)	Icon identifier
	is_active	BOOLEAN	Visibility status
`suppliers`	id	INT PK AUTO_INC	Primary key
	name	VARCHAR(200)	Business name
	slug	VARCHAR(200) UNIQUE	URL-friendly slug
	description	TEXT	About the supplier
	email	VARCHAR(100)	Contact email
	phone	VARCHAR(50)	Phone number

Table	Column	Type	Description
	mobile	VARCHAR(50)	Mobile number
	website	VARCHAR(255)	Website URL
	contact_person	VARCHAR(200)	Contact person name
	contact_first_name	VARCHAR(100)	First name
	contact_last_name	VARCHAR(100)	Last name
	address	VARCHAR(500)	Full address
	street	VARCHAR(200)	Street address
	city	VARCHAR(100)	City
	country	VARCHAR(100)	Country
	commercial_register	VARCHAR(100)	Commercial register #
	tax_number	VARCHAR(100)	Tax/VAT number
	currency	VARCHAR(10)	Default currency
	opening_balance	DECIMAL(12,2)	Opening balance
	rating	DECIMAL(3,2)	Average rating
	is_active	BOOLEAN	Active status
`products`	id	INT PK AUTO_INC	Primary key
	name	VARCHAR(200)	Product name
	slug	VARCHAR(200) UNIQUE	URL slug
	description	TEXT	Product description
	price	DECIMAL(10,2)	Price in USD
	category_id	INT FK	Ref: categories(id)
	supplier_id	INT FK NULL	Ref: suppliers(id)
	stock	INT	Quantity in stock
	sku	VARCHAR(50) UNIQUE	Stock Keeping Unit
	features	JSON	Feature list
	specifications	JSON	Key-value specs
	image	VARCHAR(255)	Image path
	rating	DECIMAL(3,2)	Average rating
	is_featured	BOOLEAN	Featured flag
	is_active	BOOLEAN	Active/deleted
`orders`	id	INT PK AUTO_INC	Primary key
	user_id	INT FK	Ref: users(id)
	total_amount	DECIMAL(10,2)	Order total
	status	VARCHAR(20)	pending/completed/etc
	shipping_address	TEXT	Delivery address
	payment_method	VARCHAR(50)	Payment type
`order_items`	id	INT PK AUTO_INC	Primary key
	order_id	INT FK	Ref: orders(id) CASCADE
	product_id	INT FK	Ref: products(id)
	quantity	INT	Item quantity
	price	DECIMAL(10,2)	Price at purchase time
`reviews`	id	INT PK AUTO_INC	Primary key
	product_id	INT FK	Ref: products(id)
	user_id	INT FK	Ref: users(id)
	rating	INT	1–5 star rating
	comment	TEXT	Review text
	created_at	TIMESTAMP	Review date

6.4 Indexes and Performance

The database employs the following indexing strategy:

- **Primary indexes** on all `id` columns (clustered).
- **Unique indexes** on `email`, `username`, `slug`, `sku`.
- **Foreign key indexes** on `category_id`, `supplier_id`, `user_id`, `order_id`, `product_id`.
- **Composite indexes** on `is_active`, `is_featured`, `status`, `price`.
- **Full-text index** on `products(name, description, sku)` for search.
- **Connection pooling** with 10 concurrent connections via `mysql2`.

6.5 Auto-Migration

The application implements an auto-migration system in `config/db.js`:

- On startup, `CREATE TABLE IF NOT EXISTS` ensures all tables exist.
- `ALTER TABLE` commands add new columns to existing tables (e.g., `supplier_id` on products).
- Errors for already-existing columns are silently ignored, making migrations idempotent.

Chapter 7 API Design

7.1 API Overview

The backend exposes a RESTful API at `http://localhost:5000/api/`. All endpoints return JSON responses. Authentication is handled via JWT Bearer tokens in the `Authorization` header.

7.2 Sequence Diagrams

The following sequence diagrams illustrate key interaction flows.

7.2.1 Main Sequence Diagram

Figure 7.1 presents the main purchase sequence for the PC Store workflow.

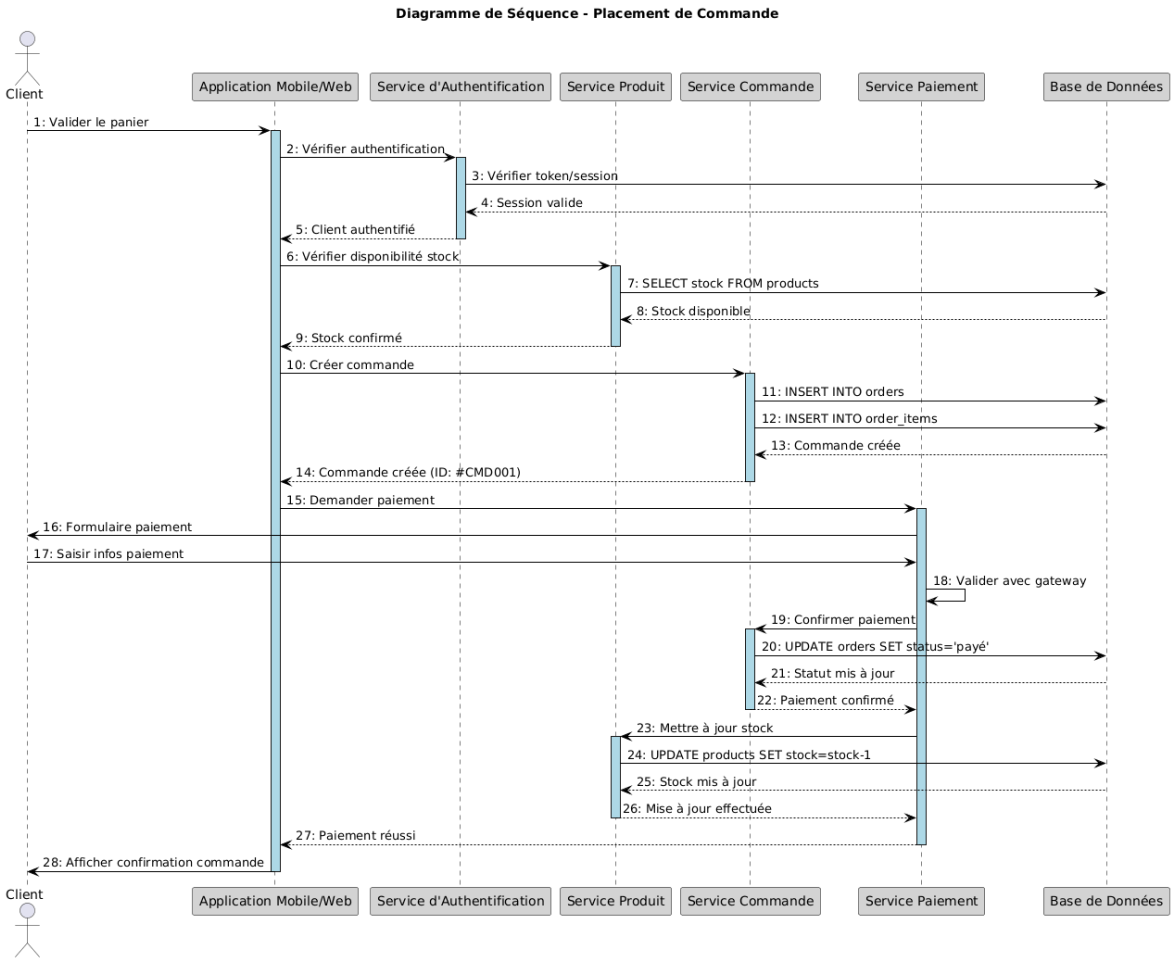

Figure 7.1: Sequence Diagram – PC Store Main Flow

7.2.2 User Registration Flow

Figure 7.2 shows the complete user registration sequence, including input validation, duplicate check, password hashing, and JWT token generation.

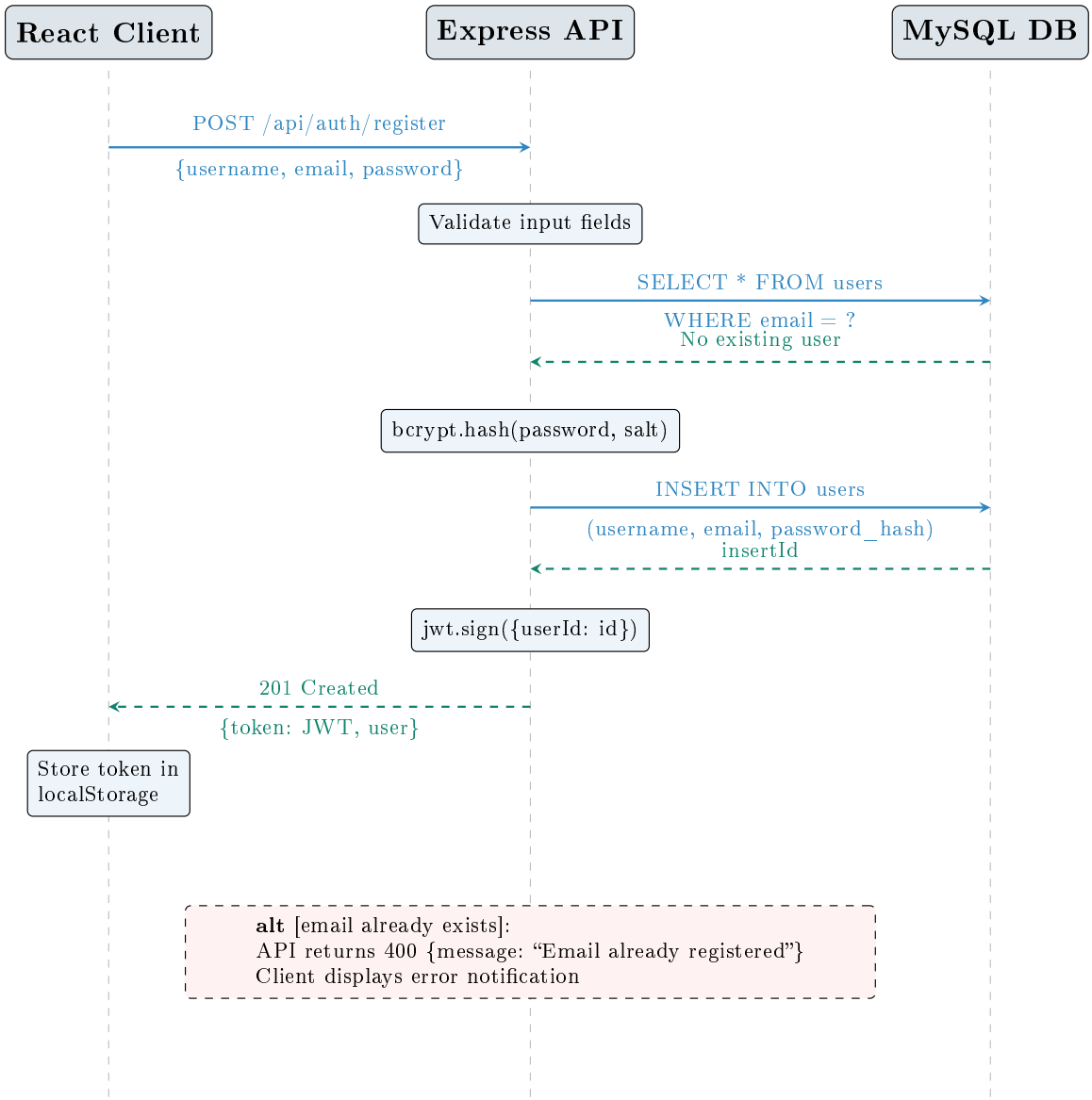

Figure 7.2: Sequence Diagram – User Registration Flow

7.2.3 Order Placement Flow

Figure 7.3 shows the checkout and order creation sequence.

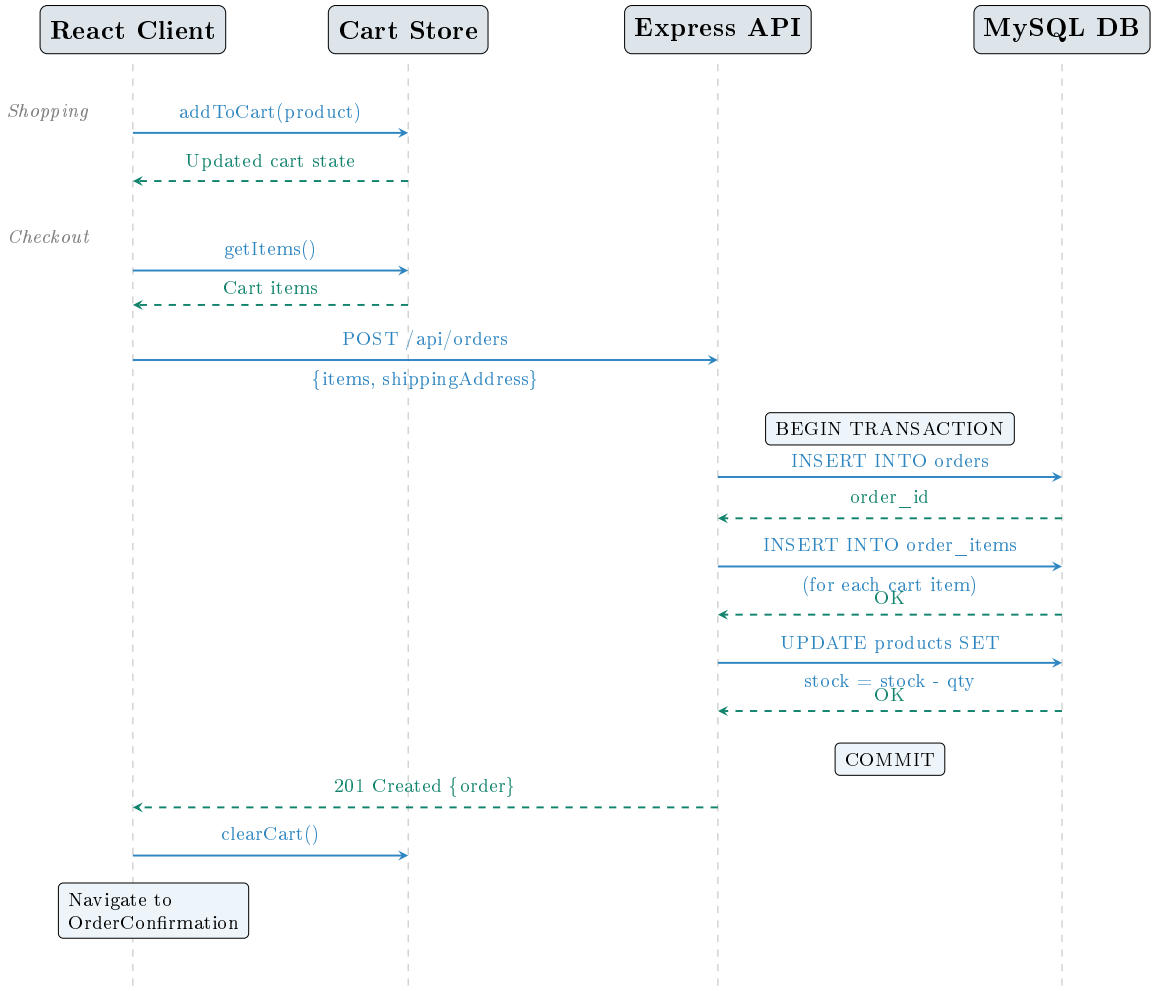

Figure 7.3: Sequence Diagram – Order Placement Flow

7.2.4 Product Review Submission

Figure 7.4 shows the sequence for an authenticated user submitting a product review, including authentication verification, review creation, and product rating recalculation.

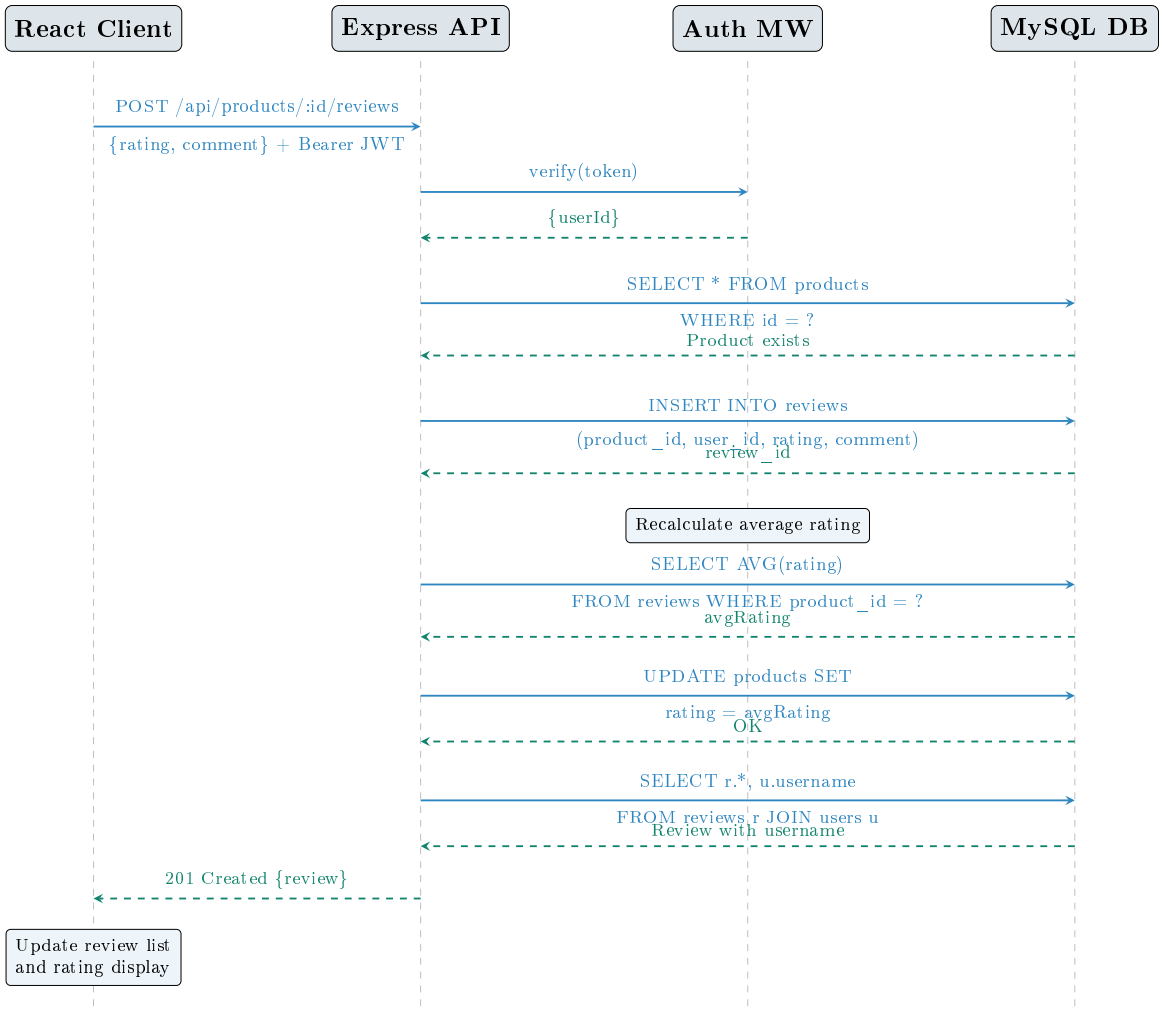

Figure 7.4: Sequence Diagram – Product Review Submission

7.2.5 Admin Product Management

Figure 7.5 shows the sequence for an admin user creating a new product through the dashboard, including both authentication and authorization checks.

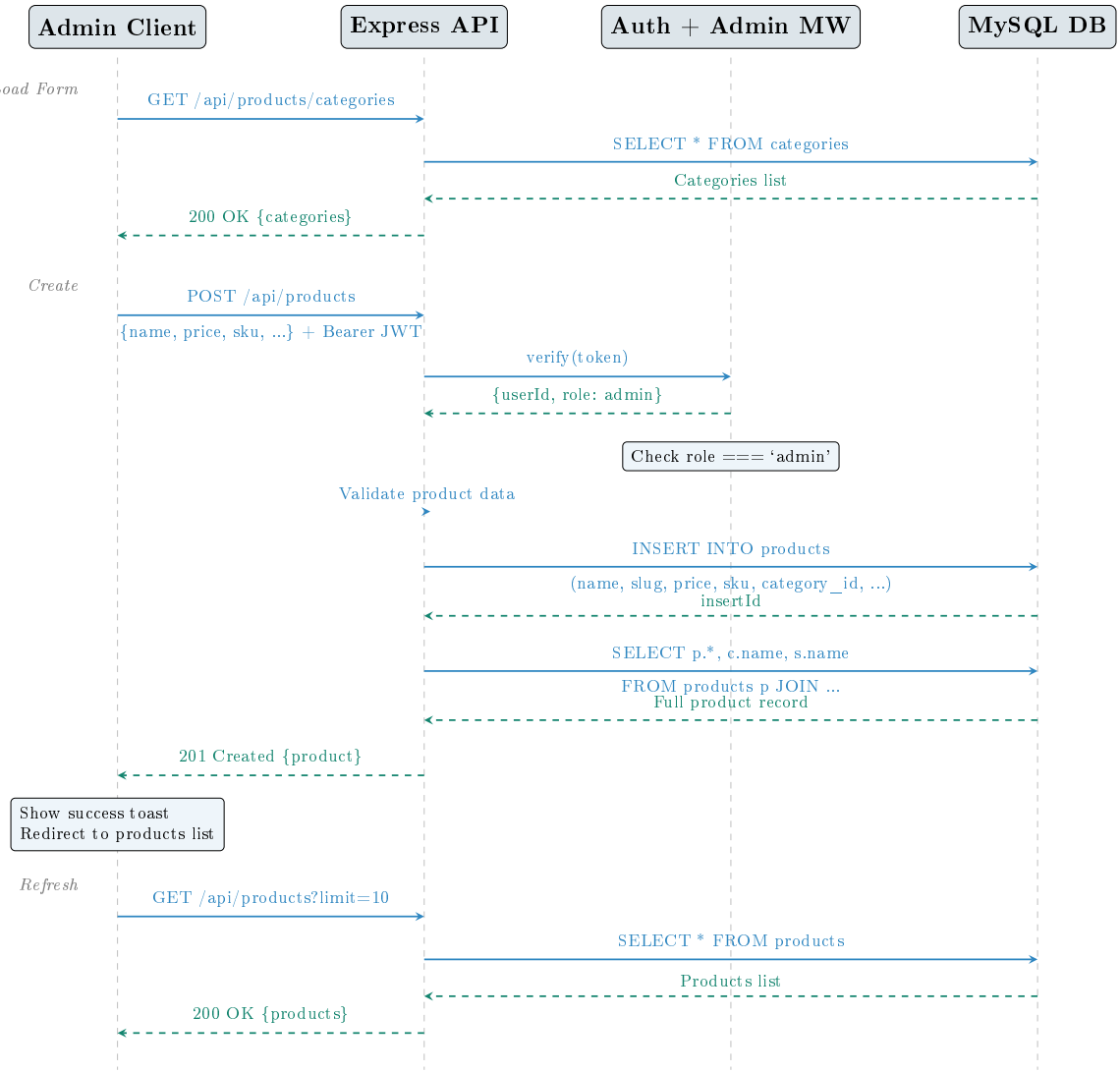

Figure 7.5: Sequence Diagram – Admin Product Creation

7.3 API Endpoints

Table 7.1: API Endpoint Reference

Method	Endpoint	Auth	Description
Authentication (/api/auth)			
POST	/auth/register	–	Register new user
POST	/auth/login	–	Login, returns JWT
GET	/auth/profile	User	Get user profile
PUT	/auth/profile	User	Update profile
Products (/api/products)			
GET	/products	–	List products (paginated)
GET	/products/featured	–	Get featured products
GET	/products/categories	–	Get all categories
GET	/products/:id	–	Get product by ID

Method	Endpoint	Auth	Description
POST	/products	Admin	Create product
PUT	/products/:id	Admin	Update product
DELETE	/products/:id	Admin	Delete product
GET	/products/:id/reviews	–	Get product reviews
POST	/products/:id/reviews	User	Add review
Orders (/api/orders)			
POST	/orders	User	Create order
GET	/orders	Admin	List all orders
GET	/orders/myorders	User	Get user's orders
GET	/orders/:id	User	Get order details
PUT	/orders/:id/status	Admin	Update order status
PUT	/orders/:id/confirm	Admin	Confirm order
PUT	/orders/:id/reject	Admin	Reject order
Users (/api/users)			
GET	/users	Admin	List all users
GET	/users/:id	Admin	Get user by ID
PUT	/users/:id	Admin	Update user
DELETE	/users/:id	Admin	Delete user
Suppliers (/api/suppliers)			
GET	/suppliers	–	List active suppliers
GET	/suppliers/admin/all	Admin	List all suppliers
GET	/suppliers/:id	–	Get supplier details
GET	/suppliers/:id/products	–	Get supplier products
POST	/suppliers	Admin	Create supplier
PUT	/suppliers/:id	Admin	Update supplier
DELETE	/suppliers/:id	Admin	Delete supplier
Stats (/api/stats)			
GET	/stats/dashboard	Admin	Dashboard statistics

7.4 Request/Response Examples

```
// POST /api/auth/login
// Request Body:
{
  "email": "user@example.com",
  "password": "securePassword123"
}

// Response (200 OK):
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "username": "john_doe",
    "email": "user@example.com",
    "role": "user"
  }
```

```
}
```

Listing 7.1: Login API – Request and Response

```
// GET /api/products?limit=10&offset=0&sortBy=price&sortOrder=ASC
// Response (200 OK):
{
  "products": [
    {
      "id": 1,
      "name": "CyberStation Pro X1",
      "slug": "cyberstation-pro-x1",
      "price": 2999.99,
      "category_name": "Gaming Desktops",
      "supplier_name": "NVIDIA",
      "supplier_id": 1,
      "stock": 25,
      "rating": 4.80,
      "features": ["RTX 5090", "Intel i9-14900K"],
      "specifications": {"CPU": "Intel i9", "GPU": "RTX 5090"}
    }
  ]
}
```

Listing 7.2: Products API – List Response

7.5 Error Handling

All API errors follow a consistent format:

```
// 400 Bad Request
{ "message": "Product name is required" }

// 401 Unauthorized
{ "message": "Not authorized, token expired or invalid" }

// 403 Forbidden
{ "message": "Not authorized as admin" }

// 404 Not Found
{ "message": "Product not found" }

// 500 Internal Server Error
{ "message": "Server error", "error": "details..." }
```

Listing 7.3: API Error Response Format

Chapter 8 User Interface

8.1 Design Philosophy

The application features a cyberpunk-themed dark UI design with the following characteristics:

- **Color Palette:** Dark backgrounds (`zinc-950`, `zinc-900`) with orange-400/500 accents and emerald status indicators.
- **Typography:** Bold, uppercase headers with tracking for a futuristic feel.
- **Animations:** Framer Motion for page transitions, hover effects, and loading states.
- **Components:** Rounded cards (`rounded-2xl/3xl`), glassmorphism effects, gradient overlays.
- **Responsiveness:** Mobile-first approach with Tailwind breakpoints (sm, md, lg, xl).

8.2 Key Pages

8.2.1 Home Page

Features a hero section with auto-sliding background images, featured products carousel, dynamic categories grid with subcategory hover dropdowns, statistics section, and newsletter subscription.

8.2.2 Products Page

Full product catalog with:

- Grid and list view toggle
- Category filtering sidebar
- Price range filtering
- Rating filtering
- Stock status filtering
- Sort by price, name, rating, newest
- Full-text search
- Pagination

8.2.3 Product Detail Page

Displays product image, name, price, rating with stars, supplier name (linked to supplier page), description, stock status, quantity selector, add to cart button, features list, specifications table, and a paginated reviews section with star rating submission.

8.2.4 Shopping Cart

Shows cart items with quantity adjustment, item removal, subtotal calculation, and checkout button. Cart state is persisted via Zustand's `persist` middleware.

8.2.5 Admin Dashboard

Tab-based administration panel with five sections:

1. **Overview:** Statistics cards (total sales, orders, users, products), recent orders list, low stock alerts, quick action buttons.
2. **Products:** CRUD table with search, sort, pagination, edit/delete actions.
3. **Orders:** Order list with status filters, status update, confirm/reject actions.
4. **Users:** User management with role editing and account deletion.
5. **Suppliers:** Supplier table with search, sort, toggle active/inactive, edit, delete with confirmation modal.

8.2.6 Supplier Pages

- **Suppliers List:** Grid of supplier cards with search, sort, and country filtering.
- **Supplier Detail:** Full supplier info (contact, address, website), products grid with pagination.
- **Add/Edit Supplier:** Four-section form (Basic Info, Contact Person, Address, Financial & Legal).

8.3 Navigation

The `CyberNavbar` component provides:

- Desktop mega-menu with product sub-categories
- Mobile hamburger menu with animated drawer
- Language toggle button (EN ↔ AR)
- Cart icon with item count badge
- User dropdown (profile, orders, admin dashboard)
- Search bar

8.4 Footer

Global footer with:

- Newsletter subscription
- Quick links (Products, Categories, Suppliers)
- Support links
- Social media icons
- Copyright notice

Chapter 9 Testing and Evaluation

9.1 Testing Approach

The project employs multiple testing strategies:

9.1.1 API Testing

Node.js scripts for end-to-end API testing:

- `api-test.js` – General API endpoint testing
- `test-admin-login.js` – Admin authentication testing
- `test-db.js` – Database connectivity testing
- `scripts/e2e-review-http.js` – End-to-end review creation via HTTP
- `scripts/test-review.js` – Review system testing

9.1.2 Database Testing

- `scripts/list-products-direct.js` – Direct database product queries
- `scripts/create-review-direct.js` – Direct database review insertion
- `test-db.js` – Connection pool and query testing

9.1.3 Manual Testing

Comprehensive manual testing was performed on:

- User registration and login flow
- Product browsing, filtering, and searching
- Shopping cart operations (add, remove, update quantity)
- Checkout and order placement
- Review submission and display
- Admin CRUD operations for all entities
- Supplier management (create, edit, toggle, delete)
- Language switching (EN $\leftrightarrow$ AR) and RTL layout
- Responsive design on mobile, tablet, and desktop viewports

9.2 Test Results Summary

Table 9.1: Testing Results Summary

Module	Test Scope	Result
Authentication	Register, login, JWT, admin access	Pass
Products	CRUD, search, filter, pagination	Pass
Orders	Create, list, status update	Pass
Users	Admin CRUD, role management	Pass
Suppliers	CRUD, toggle, product association	Pass
Reviews	Create, list, pagination	Pass
Cart	Add, remove, persist, checkout	Pass
i18n	EN/AR translations, RTL layout	Pass
Responsive	Mobile, tablet, desktop	Pass
API Security	Auth middleware, admin guard	Pass
Database	Connection pool, migrations	Pass

9.3 Performance Evaluation

- **API Response Time:** Average <100ms for product listing queries.
- **Database Queries:** Optimized with indexes and connection pooling (10 concurrent connections).
- **Frontend Bundle:** Optimized with React production build and code splitting via React Router.
- **Image Handling:** Base64 upload with server-side file storage (max 10MB request body).

Chapter 10 Conclusion and Future Work

10.1 Conclusion

CyberStore successfully demonstrates the development of a comprehensive, full-stack e-commerce web application using modern technologies. The project achieves all stated objectives:

1. A fully functional e-commerce platform with 20+ pages and comprehensive admin tools.
2. Secure authentication with JWT and role-based access control.
3. A normalized MySQL database with 7 tables, foreign keys, indexes, and full-text search.
4. Complete supplier management system integrated with the product catalog.
5. Full bilingual support (English/Arabic) with 1,500+ translation keys and RTL layout.
6. Product review and rating system.
7. Responsive, animated UI with a cyberpunk design theme using Tailwind CSS and Framer Motion.

The three-tier architecture (React frontend, Express backend, MySQL database) provides a clean separation of concerns, making the codebase maintainable and extensible. The auto-migration system ensures database schema consistency across deployments.

10.2 Challenges Faced

- **MySQL TEXT DEFAULT limitation:** MySQL does not allow DEFAULT values on TEXT columns, requiring schema adjustments (e.g., `address VARCHAR(500)` instead of `TEXT`).
- **Express route ordering:** Static routes like `/admin/all` must be registered before dynamic `/:id` routes to prevent parameter conflicts.
- **RTL layout:** Implementing full right-to-left support required careful handling of text alignment, margins, padding, and icon positioning across all components.
- **State synchronization:** Managing cart persistence and auth state across page reloads required careful Zustand middleware configuration.

10.3 Future Work

1. **Payment Gateway Integration:** Integrate Stripe or PayPal for real payment processing.
2. **Email Notifications:** Order confirmation emails, password reset, and promotional newsletters.
3. **Real-Time Features:** WebSocket integration for live order tracking and admin notifications.

4. **Inventory Management:** Automated stock alerts and purchase order generation for suppliers.
5. **Analytics Dashboard:** Sales charts, revenue trends, and customer behavior analytics.
6. **Mobile Application:** React Native mobile app sharing the same API backend.
7. **Image CDN:** Cloud-based image storage (AWS S3 / Cloudinary) for better performance.
8. **Automated Testing:** Jest + React Testing Library for unit tests; Cypress for E2E tests.
9. **Docker Deployment:** Containerized deployment with Docker Compose.
10. **Search Engine:** Integrate Elasticsearch for advanced product search and suggestions.

References

[1] Facebook/Meta, "React – A JavaScript Library for Building User Interfaces," https://react.dev/, 2024.

[2] Express.js, "Express – Node.js Web Application Framework," https://expressjs.com/, 2024.

[3] MySQL, "MySQL 8.0 Reference Manual," https://dev.mysql.com/doc/refman/8.0/en/, 2024.

[4] React Router, "React Router v6 Documentation," https://reactrouter.com/, 2024.

[5] Tailwind Labs, "Tailwind CSS Documentation," https://tailwindcss.com/docs, 2024.

[6] Zustand, "Zustand – Bear Necessities for State Management," https://zustand-demo.pmnd.rs/, 2024.

[7] JSON Web Tokens, "Introduction to JSON Web Tokens," https://jwt.io/introduction, 2024.

[8] Framer, "Framer Motion – Production-Ready Animations for React," https://www.framer.com/motion/, 2024.

[9] Axios, "Axios – Promise Based HTTP Client," https://axios-http.com/, 2024.

[10] Lucide, "Lucide Icons – Beautiful Consistent Icons," https://lucide.dev/, 2024.

[11] Node.js Foundation, "Node.js Documentation," https://nodejs.org/en/docs/, 2024.

[12] bcryptjs, "bcrypt.js – Optimized bcrypt in JavaScript," https://www.npmjs.com/package/bcryptjs, 2024.